CaneSense AI

PRECISION IRRIGATION INTELLIGENCE

Cloud Computing Mini-Project Consolidated Use-Case Report

CaneSense AI: AI-Powered Precision Irrigation Intelligence for Sugarcane Farms

STUDENT	**Raj Tripathi**
ROLL NUMBER	**16014223062**
BATCH	**B1 AIDS**
ACADEMIC STATUS	**Final-Year Student**
PROFESSOR / EVALUATOR	**Sagar Korde**
SUBJECT / ASSIGNMENT	**Cloud Computing Experiments through Somaiya Vidyavihar AI Use Cases**
SELECTED USE CASE	**KJS-AGR-01**

Report generated 23 October 2026. All application data is synthetic. Developed with AI assistance (see Section 12).

Photo: Timothy A. Gonsalves, CC BY-SA 4.0 (Wikimedia Commons).

Contents

1. Problem statement, objectives and scope

2. Dataset generation and ML methodology

3. Architecture and technology stack

4. Application walkthrough (screenshots)

5. Experiment 2 — Docker and Docker Hub

6. Experiment 3 — Kubernetes

7. Experiment 7 — Cloud resource right-sizing

8. Experiment 8 — FinOps and instance-type comparison

9. Testing and performance results

10. Cost comparison summary

11. Limitations, privacy and future improvements

12. Acknowledgement of AI-assisted development

13. References

Appendix A. Verification status and deliverables

Appendix B. Image credits

Experiment	Status	Evidence
2 Docker / Docker Hub	**Prepared, NOT executed** (Docker not installed)	Dockerfile, .dockerignore, guide; evidence/exp2_docker/status.json
3 Kubernetes	**Prepared, NOT executed** (no kubectl / cluster)	k8s/*.yaml validated by tests; evidence/exp3_kubernetes/status.json
7 Right-sizing	**Executed** (local CPU-affinity emulation)	experiments/results/rightsizing.*; evidence/exp7_rightsizing/
8 FinOps	**Executed** (estimates from assumed prices)	experiments/results/finops.*; evidence/exp8_finops/

1. Problem statement, objectives and scope

Sugarcane is one of the most water-intensive crops. Farm managers must decide, plot by plot, when to irrigate and how much water to apply, balancing soil moisture, crop stage, evapotranspiration, rainfall forecasts, pump capacity and sensor reliability. Manual scheduling is slow and tends to over- or under-irrigate; sensor outages and restricted water availability make it harder.

Use case KJS-AGR-01 is addressed by **CaneSense AI**, a decision-support application that predicts irrigation urgency with machine learning, converts the prediction into an actionable recommendation (action, water quantity, duration), and lets an operator test scenarios such as a dry spell or a sensor failure.

Objectives

- Build a working, deployable agricultural-analytics application with a genuine scikit-learn model trained and evaluated on a proper train/test split.
- Keep the recommendation logic separate, transparent and traceable to the current data.
- Provide deterministic scenario simulation, exports (PDF, CSV, Excel) and a polished, accessible interface that works offline.
- Package and operate the application with the four assigned cloud experiments: Docker (2), Kubernetes (3), right-sizing (7) and FinOps (8), reporting verification status honestly.

Scope and boundaries

- In scope: synthetic estate of 28 plots in 6 fictional villages, urgency classification, rule-based recommendations, scenarios, reporting, cloud experiments.
- Out of scope: real sensors, real farm data, weather APIs, hardware control, field validation of the model, production cloud hosting.
- All data and labels are synthetic; nothing in this report claims real-world accuracy.

2. Dataset generation and ML methodology

2.1 Synthetic dataset

`canesense/data.py` simulates 28 plots over 150 days (4,200 plot-day rows, seed 42). Each plot has a crop stage, variety, soil type with a water-holding factor, irrigation system (drip / sprinkler / furrow), area, synthetic coordinates and a sensor with battery level. A daily water balance lowers soil moisture with evapotranspiration, raises it with village-level rain events and with each plot's irrigation cycle. The final day is the current snapshot; the last 21 days feed trend charts. Validation rejects out-of-range values, duplicate IDs, unknown stages and missing columns, while a missing moisture reading (sensor outage) is handled explicitly.

2.2 Labels

Urgency (Low / Medium / High) is derived from an agronomic water-stress score (crop-stage trigger, ET0, temperature, rain, days since irrigation, soil capacity) plus Gaussian noise. These are **synthetic ground-truth labels**, not field observations.

2.3 Model and evaluation

A RandomForest classifier (300 trees, depth 12, balanced class weights) uses 10 features. A stratified 75/25 split gives 3,150 training and 1,050 test rows.

Metric (held-out test set)	Value
Accuracy	0.927
Precision (macro)	0.875
Recall (macro)	0.918
F1 (macro)	0.895
5-fold CV macro-F1	0.840 ± 0.017

Class	Precision	Recall	F1	Support
Low	0.987	0.938	0.962	737
Medium	0.754	0.864	0.805	184
High	0.885	0.953	0.918	129

Confusion matrix (rows = true, columns = predicted; Low, Medium, High): [[691, 46, 0], [9, 159, 16], [0, 6, 123]]. Soil moisture is the most important feature (0.61).

Interpretation: the scores show how well the model recovers the synthetic labelling rule from noisy inputs. They are not evidence of accuracy on real sugarcane farms.

2.4 Recommendation engine and scenarios

A separate rule engine turns the predicted urgency into an action: *Sensor readings need review* (offline sensor), *Monitor conditions*, *Rainfall adjustment recommended* (forecast ≥ 10 mm), *Irrigate soon*, or *Restricted water availability* when the shared daily budget is exhausted. Water depth = (trigger + 6 pts − moisture) × root-zone depth, capped at 75 mm, minus 70% of forecast rain, divided by system efficiency; volume = depth × area × 10 m³; duration = depth ÷ application rate.

Seven deterministic scenarios transform the baseline and re-run the model and engine for every plot (computed results below).

Scenario	High-urgency	Need irrigation	Demand (m³)	Deferred
Normal Conditions	10	13	16,807	0
Simulate Dry Spell	21	21	35,436	10
Simulate Heatwave	12	15	21,271	2
Rain Expected	10	18	12,928	0
Low Soil Moisture	27	27	46,754	13

Scenario	High-urgency	Need irrigation	Demand (m³)	Deferred
Sensor Failure	10	9	10,232	0
Water Availability Restricted	10	13	16,807	7

3. Architecture and technology stack

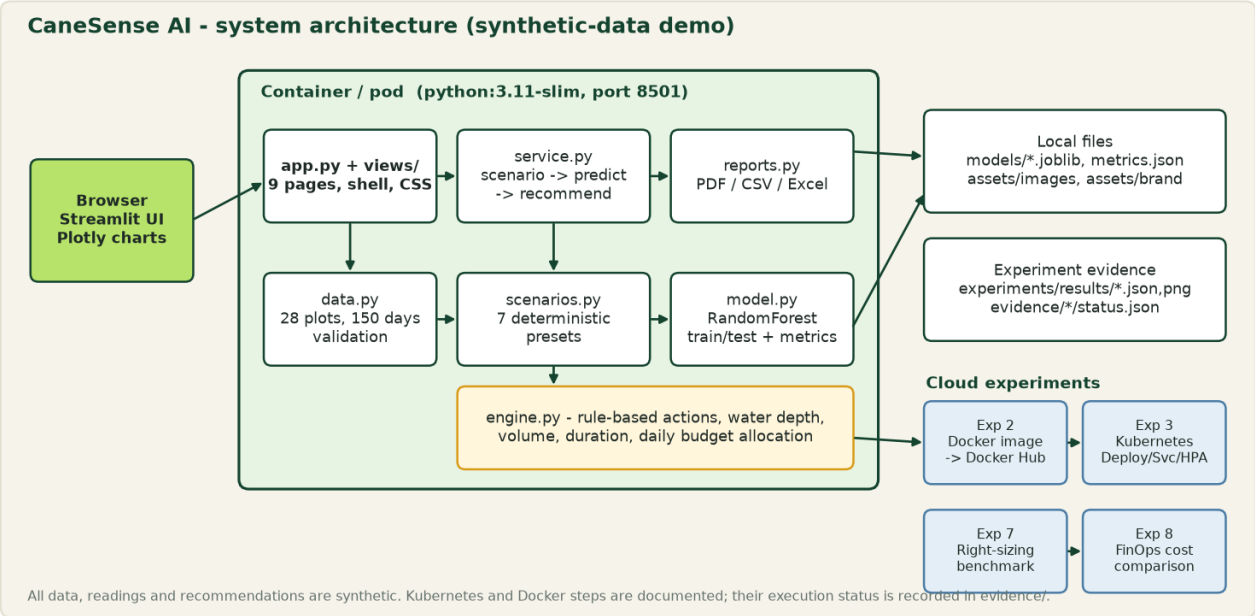


Figure 1. System architecture (editable source: docs/architecture.mmd)

Layer	Technology
UI	Streamlit 1.54 with a custom CSS design system, st.navigation, Plotly 6
ML	scikit-learn 1.9 RandomForest, joblib persistence
Data	pandas / NumPy synthetic simulation, validation layer
Reports	ReportLab (PDF), XlsxWriter (Excel), CSV, Matplotlib charts
Packaging	Docker (python:3.11-slim, non-root, health check), Kubernetes manifests (Deployment, Service, HPA)
Experiments	psutil CPU affinity benchmark, FinOps cost model
Quality	pytest (23 tests), Playwright browser checks

The application is stateless apart from per-session UI state. The model is persisted in `models/` and, in Docker, trained at build time so it always matches the image's scikit-learn version.

4. Application walkthrough (screenshots)

All screenshots below were captured from the running application by `scripts/capture_report_shots.py` (Playwright, 1440×1000).

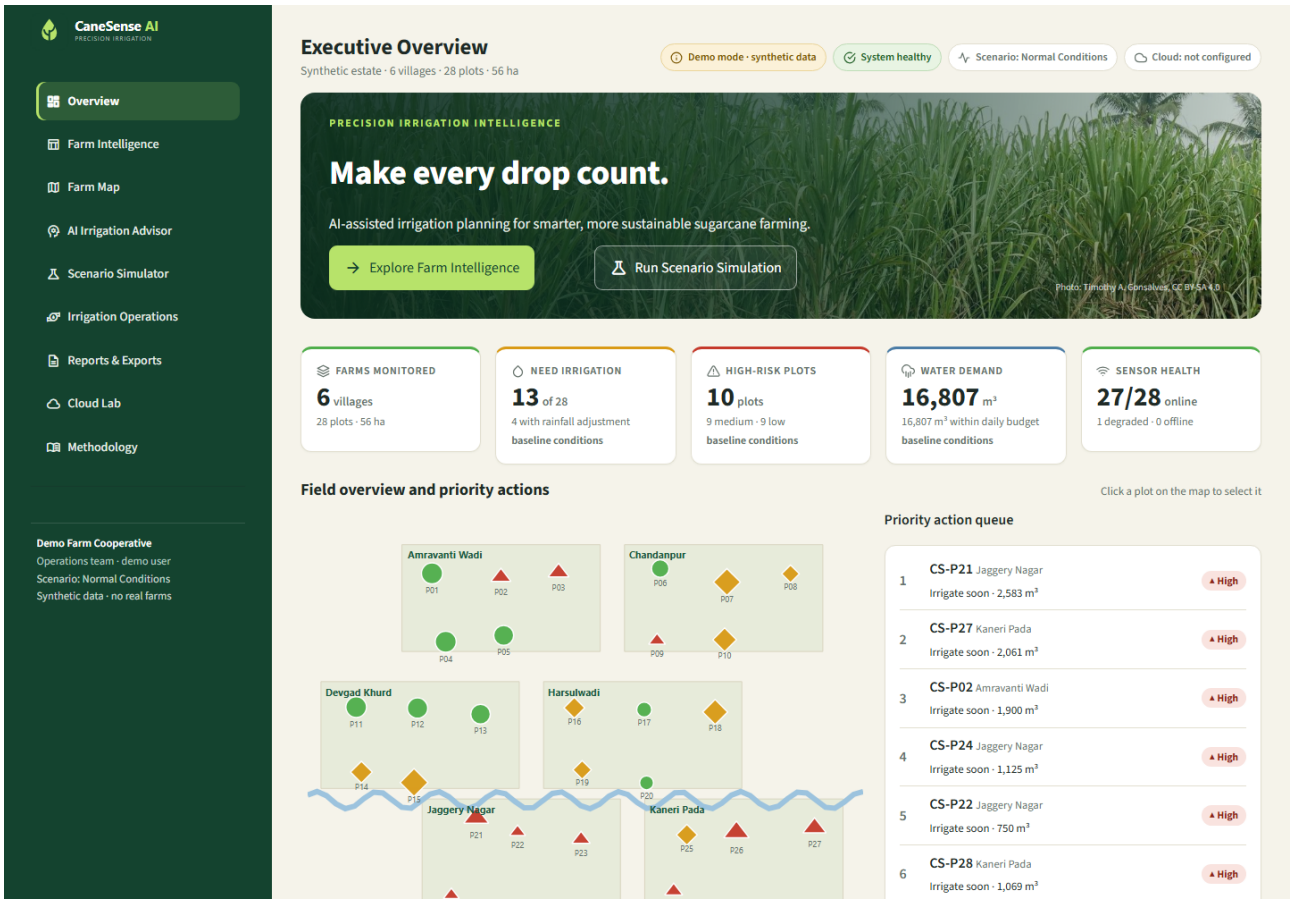


Figure 2. Overview: hero, executive KPIs computed from the dataset, field map and priority queue (Normal Conditions).

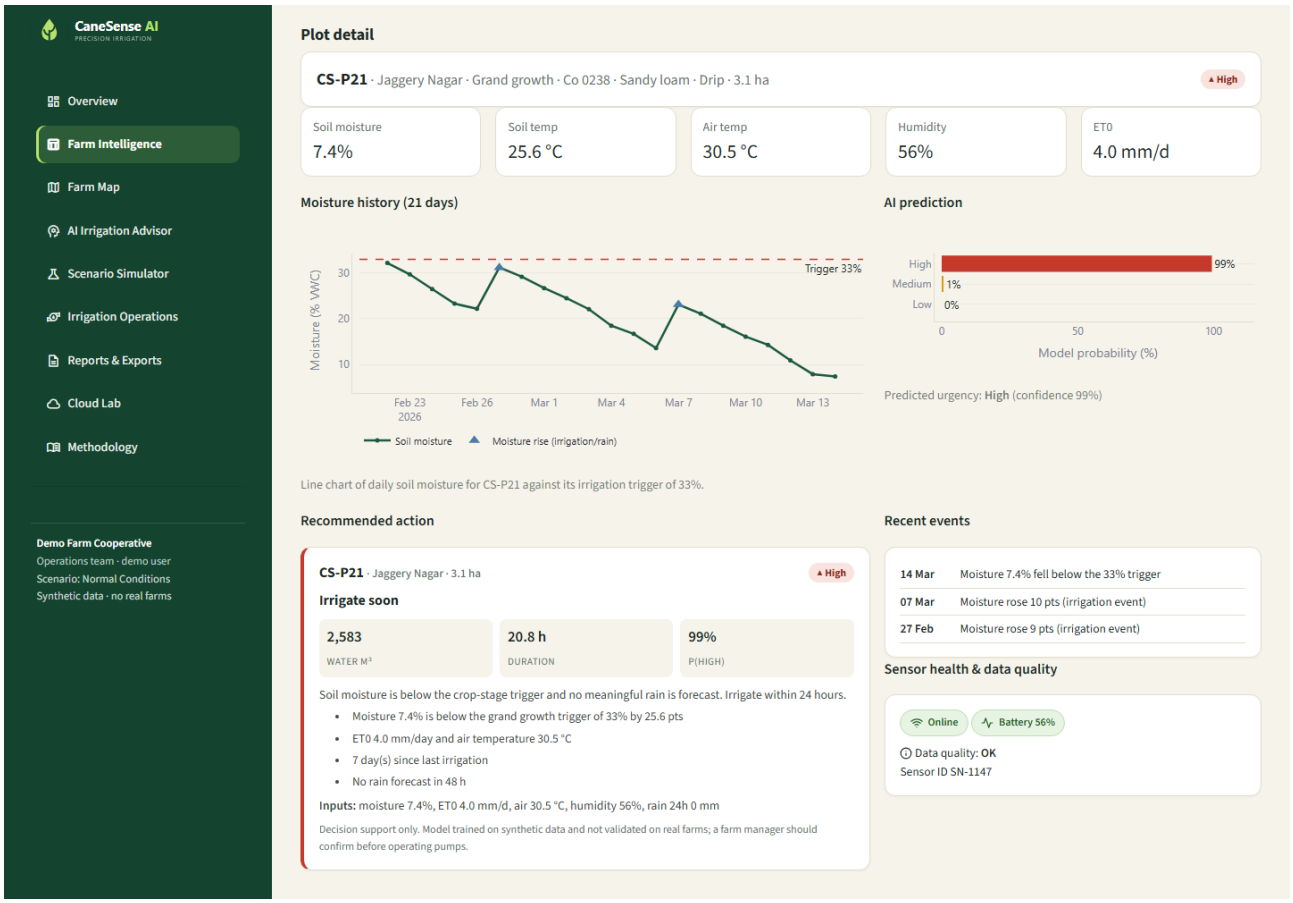


Figure 3. Farm Intelligence: plot detail with moisture history, AI prediction, recommendation, events and sensor health.

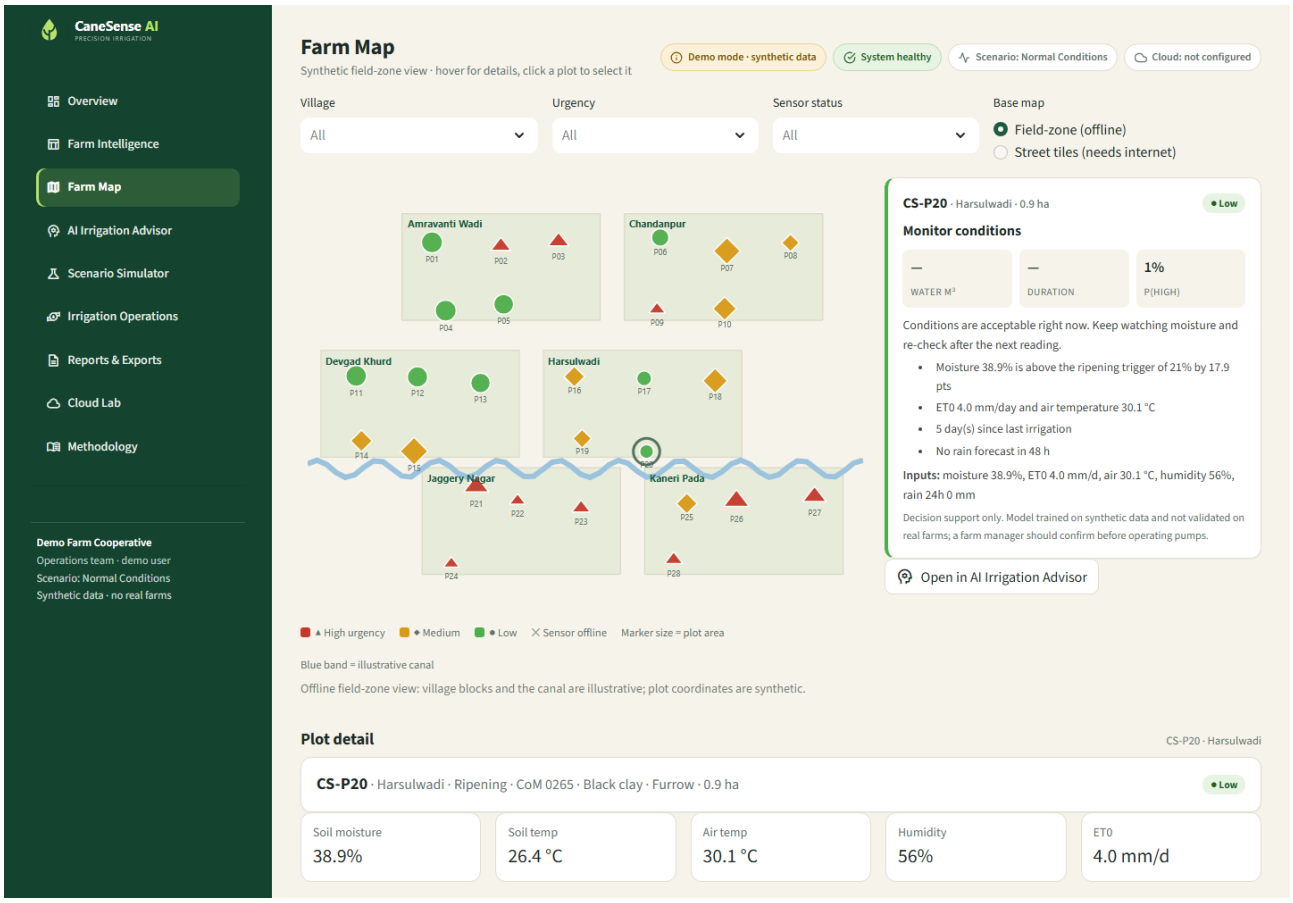


Figure 4. Farm Map: offline field-zone view with risk shapes and colours, selected plot and linked recommendation.

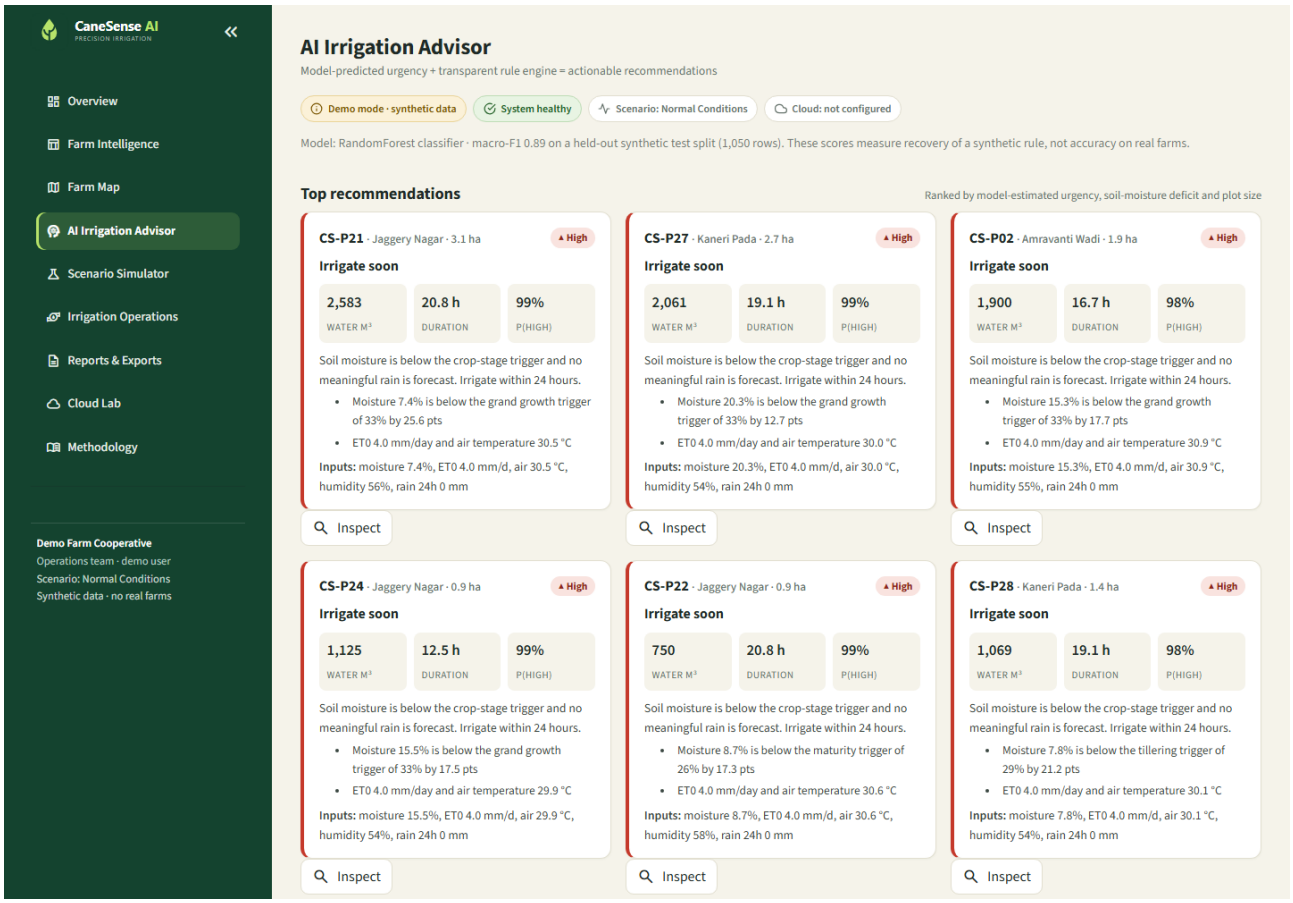


Figure 5. AI Irrigation Advisor: recommendation cards with water volume, duration, probability and reasoning.

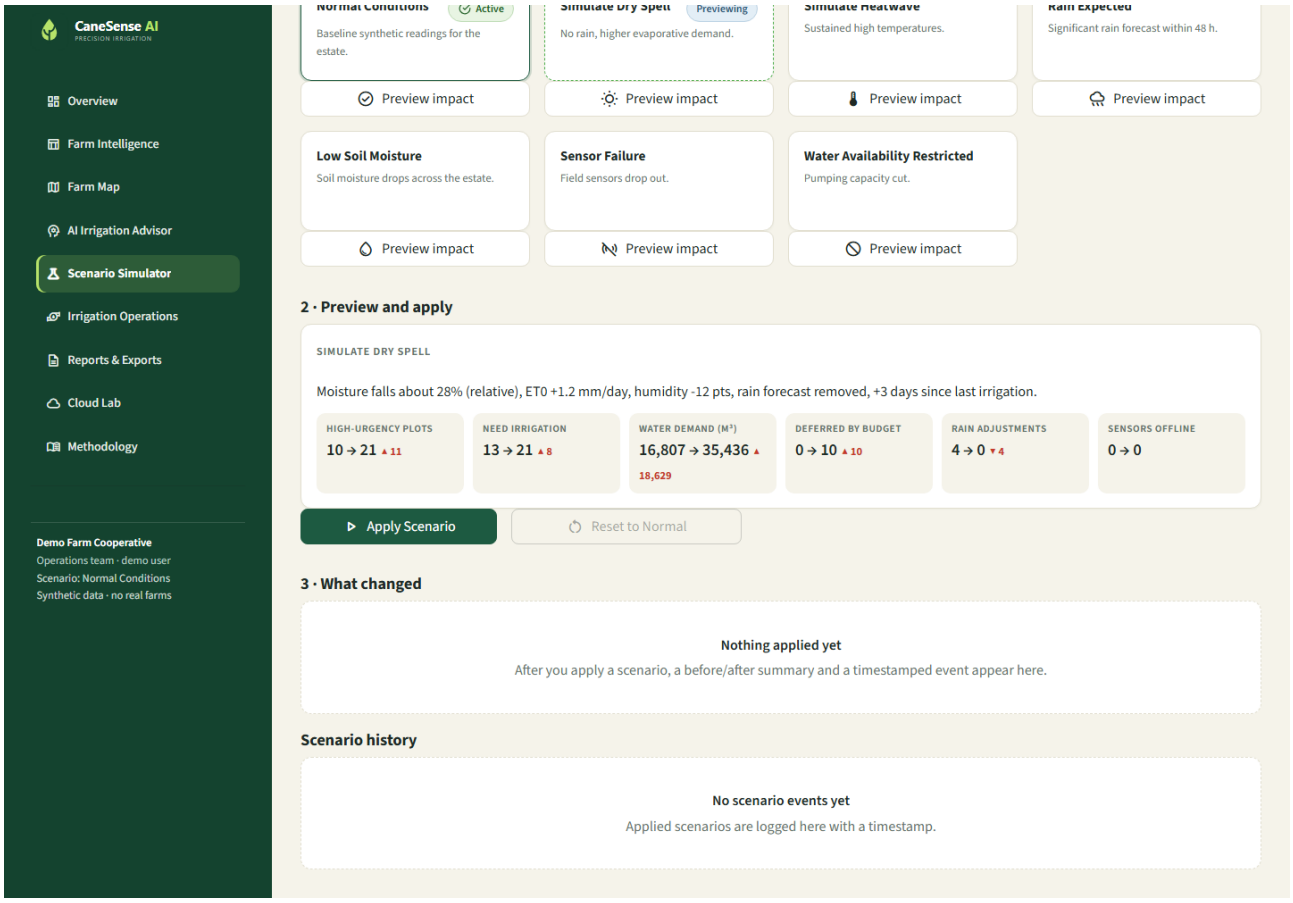


Figure 6. Scenario Simulator: preview of the Dry Spell impact before applying.

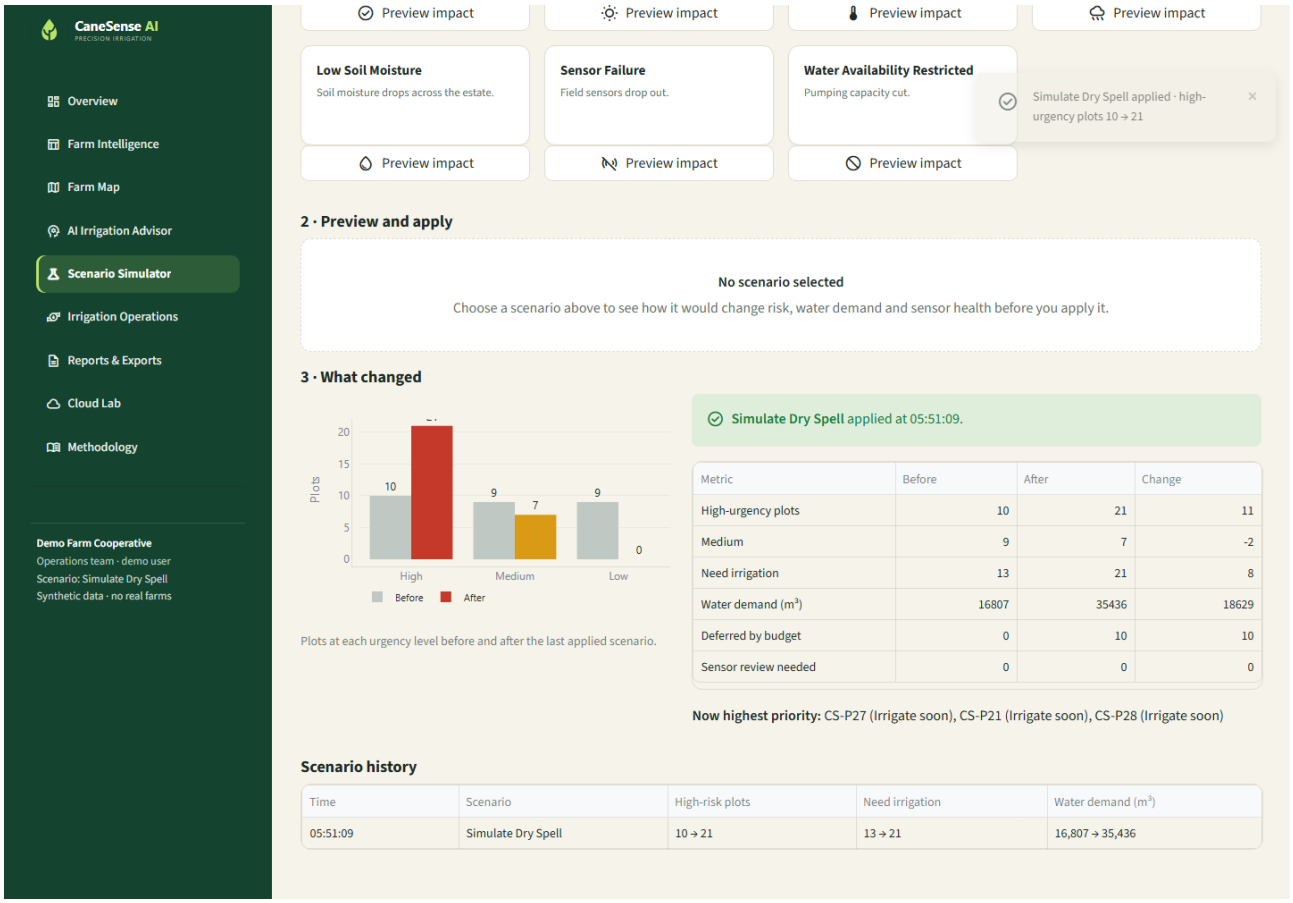


Figure 7. After Apply Scenario: before/after summary and timestamped history.

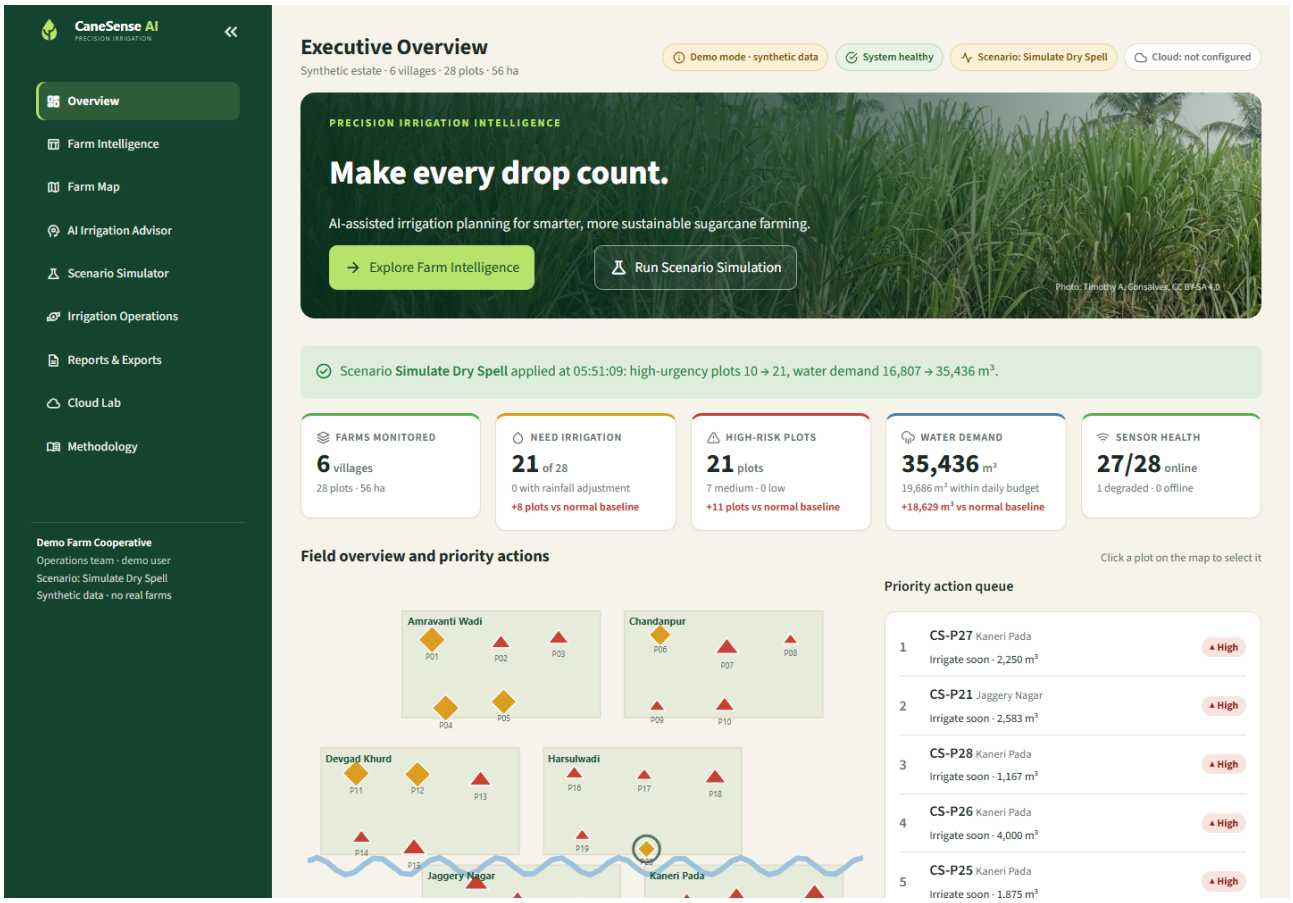


Figure 8. Overview recalculated under Dry Spell: KPIs, map colours and queue changed.

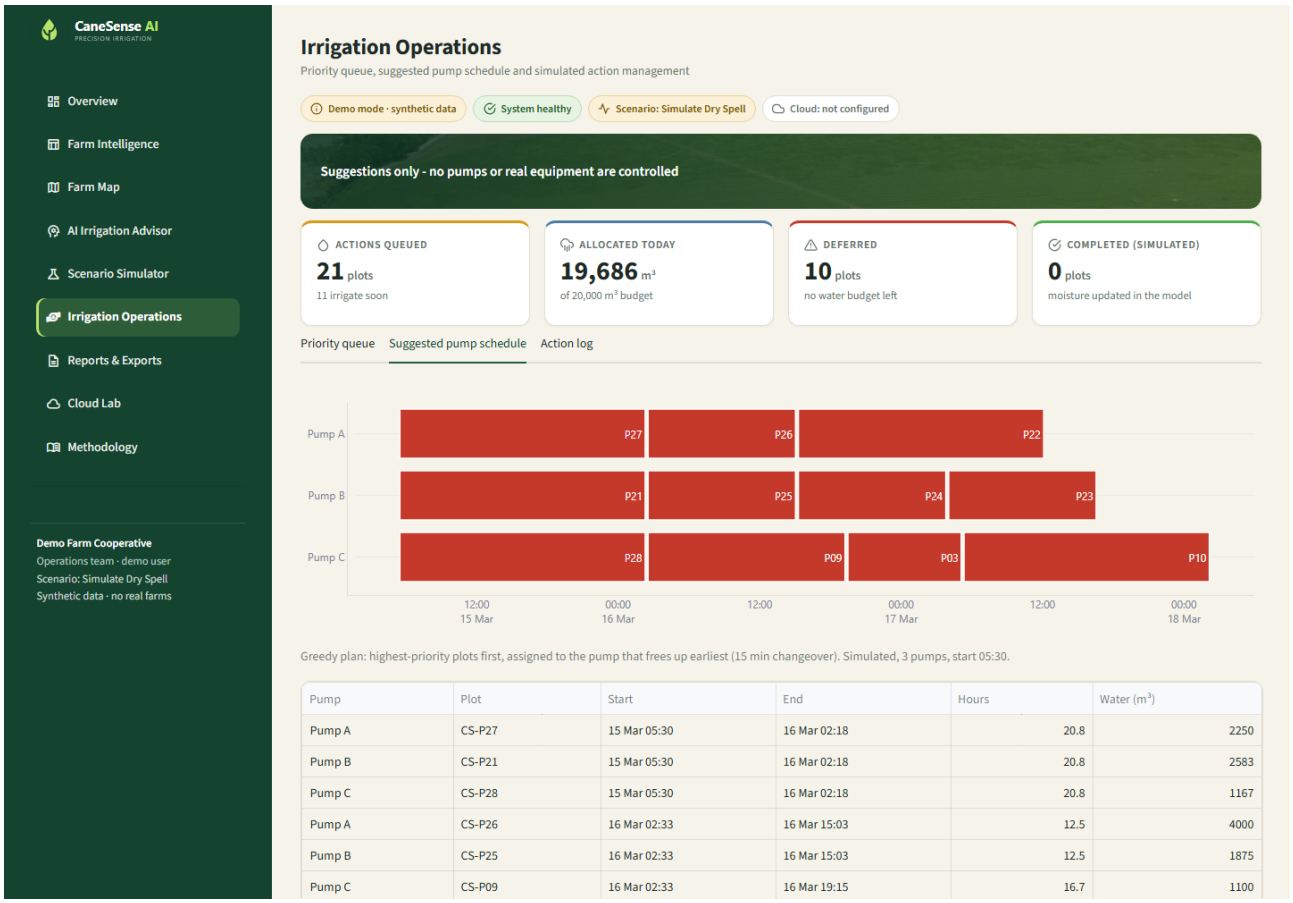


Figure 9. Irrigation Operations: suggested pump schedule (simulated).

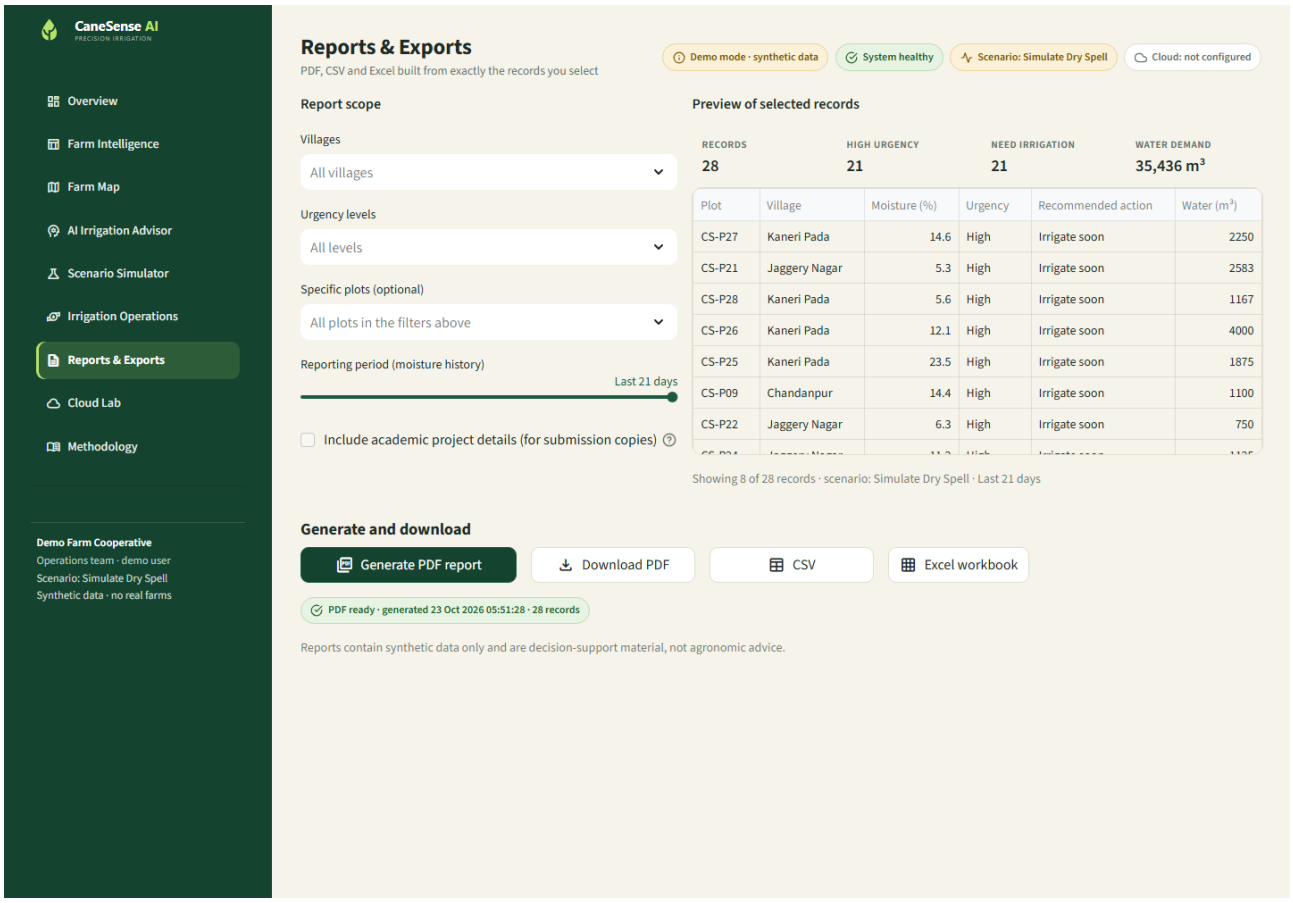


Figure 10. Reports & Exports: scope, preview and PDF/CSV/Excel downloads.

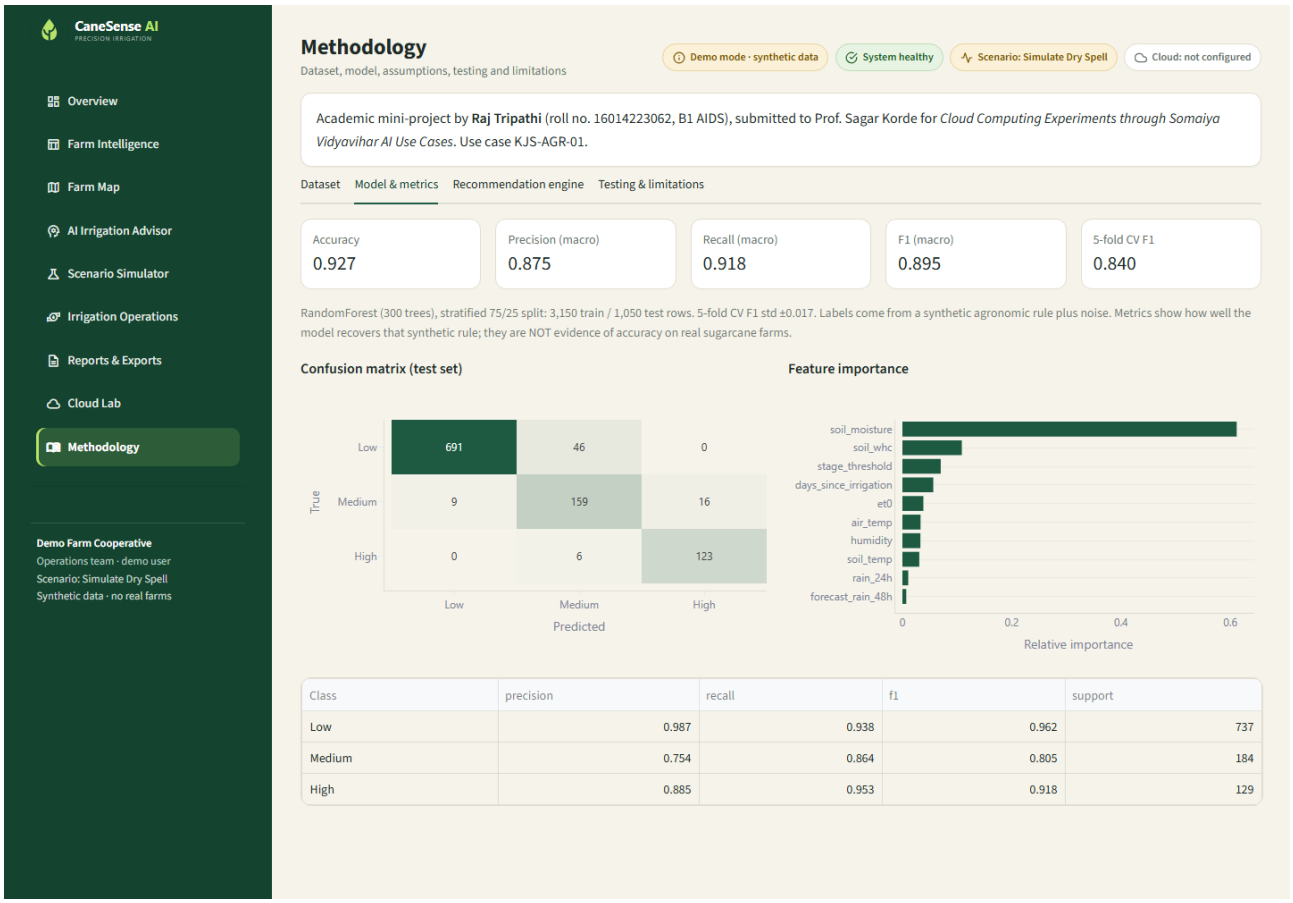


Figure 11. Methodology page: computed metrics, confusion matrix and feature importance.

Sample generated report

A farm-operations PDF generated by the application (Dry Spell scope) is provided as `outputs/CaneSense_Farm_Operations_Report_DrySpell.pdf`; a Normal-conditions copy is `CaneSense_Farm_Operations_Report_Normal.pdf`. They contain farm data only, with a branded header, charts, repeated-header tables, page numbers and a synthetic-data disclaimer.

5. Experiment 2 — Docker and Docker Hub

Verification status: NOT EXECUTED. Docker is not installed on the development machine, so the image was not built, run or pushed. No logs, screenshots or Docker Hub URLs are claimed.

Work completed

- `Dockerfile` (python:3.11-slim; dependency layer cached; copies `assets/`, `canesense/`, `views/`, `models`-generating scripts; trains the model during build; non-root UID 10001; `HEALTHCHECK` on `/_stcore/health`; port 8501).
- `.dockerignore` excluding scratch files, tests, outputs, caches and old model pickles.
- Automated test confirming the Dockerfile copies the asset directories and declares the health check and non-root user.
- Static asset paths are identical locally and in the image (`assets/images`, `assets/brand`).

Commands to run (documented in docs/experiment2_docker.md)

```
docker build -t canesense-ai:1.0 .
docker run -d --name canesense -p 8501:8501 canesense-ai:1.0
curl.exe http://localhost:8501/_stcore/health
docker login
docker tag canesense-ai:1.0 <user>/canesense-ai:1.0
docker push <user>/canesense-ai:1.0
```

Conclusion: the container definition is complete and statically verified; build, run and registry push remain to be executed on a machine with Docker (outstanding manual step).

6. Experiment 3 — Kubernetes

Verification status: NOT EXECUTED. No `kubectl` or local cluster is available. Nothing was deployed and no pod output is presented.

Work completed

- `k8s/namespace.yaml`, `configmap.yaml`, `deployment.yaml` (2 replicas, rolling update with maxUnavailable 0, non-root, readiness and liveness probes, requests 250m CPU / 512Mi, limits 1 CPU / 1Gi), `service.yaml` (ClusterIP 80→8501), optional `hpa.yaml` (2–5 pods at 70% CPU).
- Resource sizing informed by Experiment 7 (peak ≈ 485 MB).
- A test parses every manifest and checks probes, requests/limits and that the Service selector matches the Deployment.

Commands to run (docs/experiment3_kubernetes.md)

```
kubectl apply -f k8s/
kubectl -n canesense rollout status deployment/canesense-ai
kubectl -n canesense port-forward svc/canesense-ai 8501:80
kubectl -n canesense scale deployment/canesense-ai --replicas=4
kubectl -n canesense delete pod <pod>     # self-healing check
```

Design note: Streamlit keeps session state per pod, so multiple replicas need session affinity for a seamless demo.
Conclusion: manifests are complete and statically checked; deployment, scaling and recovery remain to be executed (outstanding manual step).

7. Experiment 7 — Cloud resource right-sizing

Verification status: EXECUTED on the development machine. This is a local emulation of vCPU profiles using CPU affinity, not a cloud-instance measurement.

Workload: CaneSense analytics pipeline: simulate 6 synthetic estates (25,200 plot-day rows), label, train RandomForest (300 trees), batch inference on all rows, 20x recommendation-engine runs. Fixed seed. **Host:** Windows 10, Intel64 Family 6 Model 197 Stepping 2, GenuineIntel, 16 logical CPUs, 31 GiB RAM, Python 3.11.0

Method: Each profile = fresh subprocess pinned to N logical CPUs via OS affinity (+ BLAS/OpenMP thread caps), 3 repetitions, median reported. Peak memory = max peak working set.

Profile	Median (s)	Min–max (s)	Speed-up	Efficiency	Peak memory (MB)	Training (s)
1 vCPU	10.97	10.84–11.41	1.00×	100%	483	9.44
2 vCPU	6.89	6.86–6.97	1.59×	80%	485	5.26
4 vCPU	4.97	4.82–4.98	2.21×	55%	486	3.37
8 vCPU	4.18	4.09–4.18	2.63×	33%	486	2.54

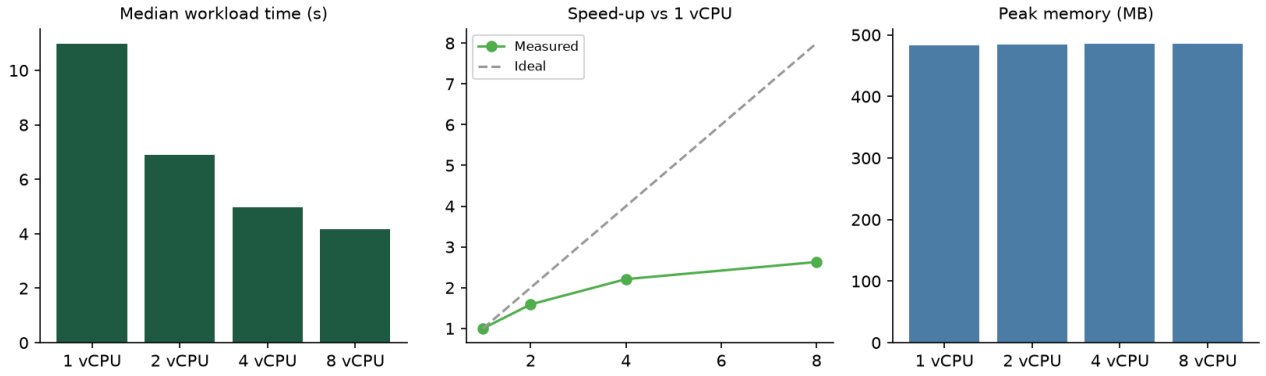


Figure 12. Measured runtime, speed-up and peak memory (experiments/results/rightsizing.png).

Memory scaling (2 vCPU): 4,200 rows → 226 MB; 12,600 rows → 334 MB; 25,200 rows → 484 MB.

Findings

- Only model training parallelises; the single-threaded data simulation limits speed-up to 2.6× at 8 vCPUs (efficiency falls from 80% at 2 vCPUs to 33% at 8).
- Memory is independent of CPU count and grows with data volume; the workload needs roughly 0.5 GB, so instances of at least 1 GiB with headroom are required.
- Best balance: 2 vCPU with 1–2 GiB (80% efficiency, 63% of the single-CPU time). 4 vCPU saves a further 1.9 s for double the CPU; 8 vCPU is poor value.

Limitations: affinity on one machine does not model a hypervisor, burst credits or a different CPU; memory was measured, not capped; the container-enforced variant (`scripts/docker_benchmark.ps1`) was not executed.

8. Experiment 8 — FinOps and instance-type comparison

Verification status: EXECUTED as a calculation. All monetary values are **estimates** from assumed public list prices, not actual charges; no cloud account was used.

ESTIMATES ONLY. Hourly prices are assumed on-demand list prices (Linux, US region), not live billing data. Runtimes are the local measurements for the matching vCPU count; real instances will differ (CPU model, burst credits on t3). Memory fit rule: measured peak 486 MB x 1.5 <= 85% of instance memory. Always-on = 730 h/month; workload-based = 24 runs/day x 30 days billed per second of run time (an idealised on-demand batch model). Storage, network, support and taxes excluded.

Instance	vCPU	GiB	$/h (assumed)	Run (s)	$/month always-on	$ per 1,000 runs	Fits memory
t3.micro	2	1	0.0104	6.89	7.59	0.020	yes
t3.small	2	2	0.0208	6.89	15.18	0.040	yes
t3.medium	2	4	0.0416	6.89	30.37	0.080	yes
c6i.large	2	4	0.0850	6.89	62.05	0.163	yes
m6i.large	2	8	0.0960	6.89	70.08	0.184	yes
c6i.xlarge	4	8	0.1700	4.97	124.10	0.235	yes
c6i.2xlarge	8	16	0.3400	4.18	248.20	0.394	yes

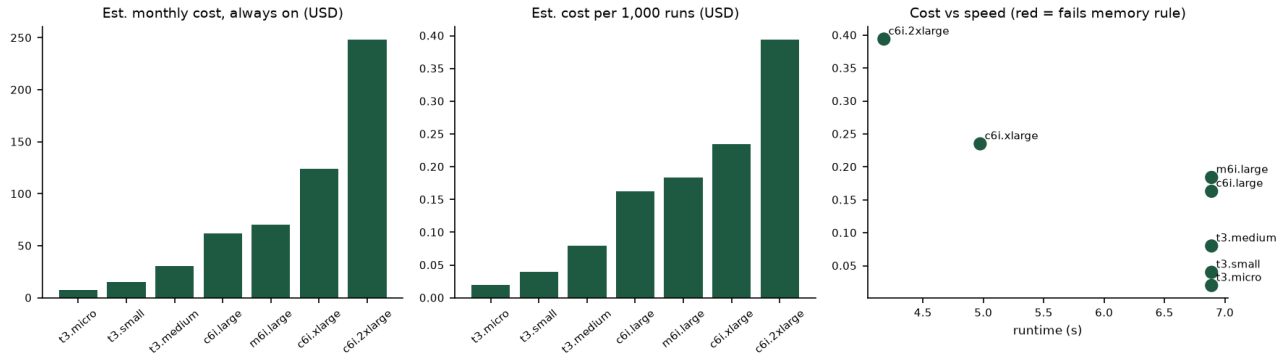


Figure 13. Estimated monthly cost, cost per 1,000 runs, and cost versus speed (experiments/results/finops.png).

For this workload the cheapest suitable choice is t3.micro (est. $0.02/1,000 runs); c6i.2xlarge is fastest (4.177 s) but costs more per run because speed-up is sub-linear.

At 24 runs per day the workload needs only seconds of compute per day, so an always-on instance is almost entirely idle; a scheduled batch job or the smallest suitable instance is the cost-efficient choice. t3-family figures ignore burst-credit throttling. Estimated charges exclude storage, network, support and taxes.

9. Testing and performance results

Automated tests (`pytest`): **23 passed in 12.51s**. The full output is in `outputs/test_results.txt`.

Test module	Covers
test_data_model	Reproducible dataset, ≥ 24 plots, validation errors, stratified split and metrics, probability sums, missing-sensor imputation
test_engine_scenarios	Deterministic scenarios, Dry Spell ↑ risk/demand, Rain Expected → rain adjustment, Sensor Failure flags, budget limit, quantities, overrides
test_reports	PDF validity and scope sensitivity, optional academic block, CSV rows equal selection, Excel sheets and rows
test_deployment_files	Kubernetes manifests parse and have probes/resources, Dockerfile contents, local assets and credits, no secrets in source
test_app_smoke	Streamlit app renders the default page with no exception

Browser verification

Playwright drove the real application: all 9 pages were loaded at 1440, 1280, 1024, 768 and 390 px wide with **0 Streamlit exceptions, 0 px horizontal overflow and 0 broken images**. An end-to-end run applied the Dry Spell scenario (high-urgency plots 10 → 21, demand 16,807 → 35,436 m³), selected a plot on the map, and downloaded the PDF and Excel reports from the UI.

Performance

Model training takes ≈ 3 s at application scale and is cached in `models/`; a scenario recalculation (model inference + engine for 28 plots) is sub-second and cached per scenario. The heavy benchmark workload (25,200 rows) takes 11.0 s on 1 vCPU and 4.2 s on 8 (Section 7).

10. Cost comparison summary

Cheapest suitable instance per run: **t3.micro** (estimated); fastest: **c6i.2xlarge**. Details and assumptions are in Section 8; all values are estimates.

11. Limitations, privacy and future improvements

- **Synthetic data and labels:** no real-farm validation; metrics measure recovery of a synthetic rule.
- **Simplified agronomy:** no crop-coefficient curve, soil-water forecasting or live weather; planning constants are illustrative.
- **Unexecuted experiments:** Docker (2) and Kubernetes (3) are prepared but unverified on this machine.
- **Benchmark scope:** CPU-affinity emulation on one host; memory measured not capped; FinOps uses assumed prices.
- **Privacy:** no personal or farm data is collected; fictional villages and arbitrary coordinates; session state stays in memory.
- **Future work:** field data validation, crop-coefficient water balance, weather API, authentication, CI/CD, managed Kubernetes deployment, container-enforced right-sizing.

12. Acknowledgement of AI-assisted development

This project was developed with the assistance of an AI coding assistant (Claude, by Anthropic), which generated and refined code, tests and documentation under the student's direction. The student specified the requirements, reviewed the outputs and is responsible for the submission. Experiment results in this report were produced by running the included scripts; experiments that could not be run are labelled as not executed.

13. References

- Streamlit documentation. https://docs.streamlit.io

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Allen, R. G., Pereira, L. S., Raes, D., Smith, M. (1998). Crop evapotranspiration — Guidelines for computing crop water requirements. FAO Irrigation and Drainage Paper 56.
- Plotly Python documentation. https://plotly.com/python/
- ReportLab Open Source User Guide. https://docs.reportlab.com
- Docker documentation. https://docs.docker.com
- Kubernetes documentation. https://kubernetes.io/docs/
- FinOps Foundation. FinOps Framework. https://www.finops.org
- Wikimedia Commons (image sources and licences). https://commons.wikimedia.org

Appendix A. Verification status and deliverables

Item	Status
Application launches, 9 pages render, major workflows verified in a browser	Verified
Automated tests	23 passed in 12.51s
Experiment 2 Docker	NOT executed (Docker not installed)
Experiment 3 Kubernetes	NOT executed (no cluster)
Experiment 7 Right-sizing	Executed (local emulation)
Experiment 8 FinOps	Executed (estimates)

Deliverable	Path
Consolidated report	outputs/16014223062_Raj_Tripathi_CC_UseCase_Report.pdf
Source code ZIP	outputs/16014223062_Raj_Tripathi_CC_UseCase_Code.zip
Demo guide (optional)	outputs/16014223062_Raj_Tripathi_CC_UseCase_Demo.pdf
Farm operations reports	outputs/CaneSense_Farm_Operations_Report_DrySpell.pdf, ..._Normal.pdf
README / docs	README.md, docs/*.md
Video script / checklist	docs/demo_video_script.md, docs/live_demo_checklist.md

Manual steps outstanding

- Run Experiments 2 and 3 on a machine with Docker / Kubernetes and record evidence in `evidence/`.
- Record the video using docs/demo_video_script.md.
- Submit the files manually through the course channel; no institutional logo, signature or endorsement has been added.

Appendix B. Image credits

All photographs were retrieved through the Wikimedia Commons API (not scraped), resized/compressed locally. Licences are as reported by Commons at download time. Attribution is required for CC BY / CC BY-SA images; CC BY-SA images are shared under the same licence. Photographs are shown only as decoration; they do not depict the synthetic plots or imply real farms.

File	Author	Licence
hero_sugarcane.jpg	Timothy A. Gonsalves	CC BY-SA 4.0
field_tamilnadu.jpg	Karthik jeyaraman	CC BY 4.0
irrigation_nrcs.jpg	USDA (NRCS)	Public domain
soil_sensor.jpg	Christianjhart1	CC0

Source pages: see assets/IMAGE_CREDITS.md.